Cloud AI Architecture and Cost Optimization for MCP-Driven Text-to-SQL Python Applications

Cloud LLM API Cost Analysis and Free Tier Evaluation

Transitioning a Model Context Protocol (MCP) text-to-SQL pipeline from local hardware inference to cloud-hosted Large Language Model (LLM) APIs shifts the primary operational constraint from hardware memory limits to financial API costs and provider rate limits. For personal logging applications, home telemetry monitoring, and light automation scripts, minimizing or eliminating recurring API inference costs is the central architecture requirement. Several cloud providers offer permanent or promotional free tiers for developer API access. However, these free offerings vary significantly regarding daily quotas, per-minute request throughput, function/tool calling support, and prompt/completion context ceilings.
Evaluating cloud API options specifically for zero-cost operation reveals distinct trade-offs across major platforms:

- **Google AI Studio (Gemini API)**: Google provides the most generous zero-cost API tier for developers without requiring a credit card on file. Models like Gemini 2.0 Flash and Gemini 2.5 Flash offer a daily quota of 1,500 Requests Per Day (RPD), 15 Requests Per Minute (RPM), and 1,000,000 Tokens Per Minute (TPM). Gemini models natively support structured JSON output and function calling, making them highly effective for MCP tool invocation.
- **GroqCloud**: Groq provides ultra-fast inference powered by Language Processing Units (LPUs). Its free tier supports open-weights models such as Llama 3.3 70B Instruct and Qwen variants. While generation throughput exceeds 300 to 460 tokens per second, free tier rate limits restrict usage to model-specific daily budgets (e.g., 1,000 RPD or 200,000 to 500,000 TPM).
- **Anthropic (Claude API)**: Anthropic offers state-of-the-art tool calling and reasoning via Claude 3.5 Sonnet and Claude 3.7 Sonnet. However, Anthropic does **not** provide a permanent free API tier for automated scripts. Developer usage requires purchasing prepaid API credits or subscribing to paid plans, making it unsuitable for a strictly zero-cost deployment.
- **OpenRouter**: OpenRouter acts as an API aggregator offering access to multiple models, including specific free-tier endpoints tagged with the :free suffix. Unfunded free accounts are constrained to 50 Requests Per Day with variable rate limits and potential endpoint unavailability, making it less reliable for primary production scripting.

Provider & Model Endpoint	Credit Card Required?	Free Requests Per Day (RPD)	Free Requests Per Minute (RPM)	Free Tokens Per Minute (TPM)	Native Tool / Function Calling	Financial Cost per 1M Input/Output Tokens
Google Gemini 2.0 Flash	No	1,500	15	1,000,000	Yes (Native)	$0.00 (Free Tier)

Provider & Model Endpoint	Credit Card Required?	Free Requests Per Day (RPD)	Free Requests Per Minute (RPM)	Free Tokens Per Minute (TPM)	Native Tool / Function Calling	Financial Cost per 1M Input/Output Tokens
Google Gemini 2.5 Flash	No	1,500	10–15	250,000–1,000,000	Yes (Native)	$0.00 (Free Tier)
Google Gemini 2.0 Pro / 2.5 Pro	No	50	2	32,000	Yes (Native)	$0.00 (Free Tier)
GroqCloud (Llama 3.3 70B)	No	1,000	30	6,000–100,000	Yes (Native)	$0.00 (Free Tier)
OpenRouter (:free endpoints)	No	50	20	Uncapped / Dynamic	Variable / Limited	$0.00 (Promotional)
Anthropic Claude 3.5 Sonnet	Yes (Paid)	None (0)	N/A	N/A	Yes (Native)	$3.00 / $15.00 (Paid Only)

Text-to-SQL Benchmarking and Schema Reasoning for Cloud Models

Converting natural language questions into precise, executable SQL statements requires strong logical reasoning, understanding of relational schemas, and strict adherence to tool outputs. In inverter telemetry logs (e.g., inverter_logs.db), queries like *"yesterday, what times was my generator struggling?"* require generating filtering conditions that identify AC input frequencies operating outside standard operational thresholds (e.g., outside 49.5Hz to 50.5Hz) combined with indexed timestamp range clauses.

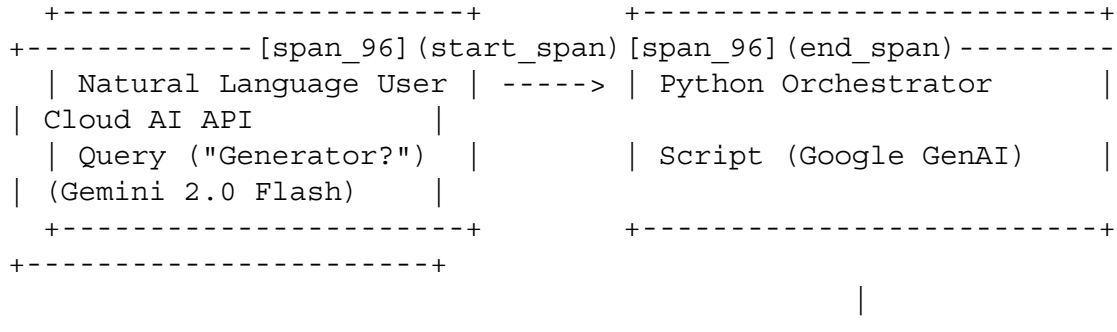
Cloud-hosted models differ in their ability to perform multi-step tool execution, SQL query synthesis, and result interpretation:

1. **Google Gemini 2.0 Flash / 2.5 Flash**: Designed for multimodal speed and low-latency agentic loops. On SQL translation tasks, Gemini Flash models reliably interpret SQLite schemas, automatically write appropriate WHERE clauses for timestamp filtering, and apply aggregate functions (AVG, MIN, MAX, COUNT) to keep returned data payloads small.
2. **Llama 3.3 70B Instruct (via GroqCloud)**: Represents one of the most capable open-weights models for general reasoning and code synthesis. When executed via Groq, it converts natural language queries to SQL with high accuracy. However, its stricter per-minute token limits on Groq's free tier require keeping context windows tightly managed.
3. **Claude 3.5 / 3.7 Sonnet (Anthropic)**: Yields industry-leading SQL generation accuracy on benchmarks like BIRD and Spider. Despite its superior accuracy, the absence of a free API tier makes it unsuitable when minimum cost is required.

Model Candidate	Free Access Tier Viability	Text-to-SQL Accuracy	Schema & Time-Series Filtering Capabilities	Context Efficiency Impact	Recommended Role in Zero-Cost Pipeline
Gemini 2.0 Flash	**Optimal** (1,500 RPD)	High	Exceptional; handles time-range calculations and frequency boundary filtering.	Excellent; strictly enforces aggregation rules to minimize token payload.	**Primary LLM Engine** for daily automated query tasks.
Llama 3.3 70B (Groq)	**High** (1,000 RPD)	Very High	Strong schema understanding and SQL syntax reliability.	Moderate; requires concise prompts to avoid hitting Groq's low TPM cap.	**Secondary / Fallback Engine** for complex analytical queries.
Gemini 2.0 Pro	**Limited** (50 RPD)	Exceptional	Superior multi-step reasoning for ambiguous natural language queries.	High; 32K TPM cap restricts frequent multi-turn conversations.	**Escalation Engine** for queries that fail on Flash models.
Claude 3.5 Sonnet	**Unviable** (Paid Only)	Industry-Leading	Flawless SQL execution and structured output formatting.	Excellent; cost scales linearly with token usage ($3/1M input).	Unsuitable due to mandatory usage costs.

Python Orchestrator Architecture and FastMCP Integration

The system deployment connects a local FastMCP server (inv_mcp.py) running on port 8000 to a lightweight Python orchestration script that communicates with cloud APIs via standard HTTP endpoints.



```
|                                                          | Tool Invocation
| Tool Call (SQL)                                          v
v                                                +--------------------------+
+-----------------------+                        |                          |
| Read-Only SQLite      |                        | FastMCP Tool Server      | <-----
| (`inverter_logs.db`)  |                        | (`execute_query()`)      |
+-----------------------+                        +--------------------------+
```

The FastMCP server (inv_mcp.py) exposes two primary read-only tools to the cloud model:

1. get_schema(): Executes SELECT sql FROM sqlite_master WHERE type IN ('table', 'index') to return complete Data Definition Language (DDL) structures and index definitions.
2. execute_query(sql_query): Executes read-only SQL queries against inverter_logs.db. A built-in security check enforces that queries must begin with SELECT, blocking modification operations (UPDATE, INSERT, DELETE, DROP).

Python Client Execution Flow

The workflow is managed by a lightweight Python client script:

```
import google.generativeai as genai
import httpx
import json

# Configure Google AI Studio Key (Free Tier)
genai.configure(api_key="YOUR_GEMINI_API_KEY")

MCP_SERVER_URL = "http://localhost:8000/mcp"

def call_mcp_tool(tool_name: str, arguments: dict = None):
    """Bridge function invoking local FastMCP HTTP tools."""
    response = httpx.post(
        f"{MCP_SERVER_URL}/tools/{tool_name}",
        json={"arguments": arguments or {}},
        timeout=10.0
    )
    return response.json()

def process_user_query(prompt: str):
    # Initialize Gemini 2.0 Flash with system instructions enforcing
aggregation
    system_instruction = (
        "You are an expert SQL assistant for inverter log telemetry. "
```

```
        "Guidelines:\n"
        "- Use aggregate functions (AVG, SUM, MIN, MAX) rather than
raw rows.\n"
        "- Always apply WHERE clauses on 'timestamp' for time
filtering.\n"
        "- Enforce frequency anomaly logic (e.g., ac_input_freq < 49.5
OR ac_input_freq > 50.5)."
    )

    model = genai.GenerativeModel(
        model_name="gemini-2.0-flash",
        system_instruction=system_instruction
    )

    # Step 1: Obtain database schema from local FastMCP tool
    schema_info = call_mcp_tool("get_schema")

    # Step 2: Request model to formulate SQL query based on user
question & schema
    llm_prompt = f"Database Schema:\n{schema_info}\n\nUser Question:
{prompt}\nGenerate the appropriate SQL query."
    response = model.generate_content(llm_prompt)

    sql_query = response.text.strip().replace("```sql",
"").replace("```", "")

    # Step 3: Execute generated SQL via FastMCP
    query_results = call_mcp_tool("execute_query", {"sql_query":
sql_query})

    # Step 4: Synthesize plain-language answer back through the model
    final_prompt = f"User Question: {prompt}\nSQL Executed:
{sql_query}\nResults: {json.dumps(query_results)}\nProvide a clear
summary."
    final_response = model.generate_content(final_prompt)

    return final_response.text
```

This Python orchestrator keeps processing light on the host machine (e.g., Raspberry Pi or local PC), utilizing negligible RAM (~50 MB) while delegating language generation to the cloud.

Rate Limit Engineering, Quota Optimization, and Resilience

Operating continuously under free cloud API quotas requires active handling of Rate Limits (RPM, TPM, RPD) to prevent 429 Resource Exhausted errors.

Free Tier Allocation and Multi-Project Scaling

Google AI Studio enforces free tier limits per Google Cloud Project rather than per individual API key. The standard free allocation for Gemini 2.0 Flash includes:

- **15 Requests Per Minute (RPM)**: Allows 1 request every 4 seconds.
- **1,000,000 Tokens Per Minute (TPM)**: Provides significant headroom for schema definitions and JSON query outputs.
- **1,500 Requests Per Day (RPD)**: Supports up to 750 complete two-pass transactions (Schema/Query Generation + Result Synthesis) per 24-hour cycle.

For applications exceeding 1,500 requests per day, developers can set up multiple distinct Google Cloud Projects within AI Studio. Implementing client-side API key rotation across separate projects scales daily capacity without incurring fees.

```
import time
from itertools import cycle

# Key rotation across separate Google Cloud Projects to multiply free
quotas
API_KEYS =
["P[span_31](start_span)[span_31](end_span)[span_41](start_span)[span_
41](end_span)[span_51](start_span)[span_51](end_span)ROJECT_A_KEY",
"PROJECT_B_KEY", "PROJECT_C_KEY"]
key_pool =
cycle(API_KEY[span_32](start_span)[span_32](end_span)[span_42](start_s
pan)[span_42](end_span)[span_52](start_span)[span_52](end_span)S)

def execute_with_backoff(prompt_func, *args, retries=3):
    """Executes API reques[span_98](start_span)[span_98](end_span)ts
with exponential backoff on
ra[span_33](start_span)[span_33](end_span)[span_43](start_span)[span_4
3](end_span)[span_53](start_span)[span_53](end_span)te limit
errors."""

[span_104](start_span)[span_104](end_span)[span_108](start_span)[span_
108](end_span)for attempt in range(retries):
        try:
            current_key = next(key_pool)

genai.configure(api_key=current[span_34](start_span)[span_34](end_span
)[span_44](start_span)[span_44](end_span)[span_54](start_span)[span_54
](end_span)_key)
            return prompt_func(*args)
        except Exception as e:
            if "429" in str(e) or "RESOURCE_EXHAUSTED" in str(e):
                wait_time = (2 ** attempt) + 1
                time.sleep(wait_time)
            else:
                raise e
```

```
    raise RuntimeError("Exceeded maximum retry attempts due to rate
limits.")
```

Context Compression and Payload Minimization

To remain comfortably within per-minute token limits (TPM) and maximize daily capacity, the Python script should apply three prompt-engineering strategies:

1. **Schema Caching**: Instead of fetching and sending the complete table schema on every request, store the DDL schema string in local file cache or RAM. Only re-query get_schema() if the database structure updates.
2. **SQL Aggregation Enforcement**: Require the LLM to write queries using SQLite aggregation primitives (AVG(), COUNT(), MIN(), MAX(), GROUP BY). Aggregating data within SQLite prevents returning thousand-row telemetry logs back to the LLM, keeping context windows small and inference fast.
3. **Strict Time Window Filtering**: Ensure the prompt instructs the model to translate relative time expressions (e.g., *"yesterday"*, *"last 3 hours"*) into explicit timestamp bounds in the SQL WHERE clause, leveraging SQLite indexes on the timestamp column.

Strategic Recommendations and Implementation Plan

For a zero-cost, cloud-assisted text-to-SQL system querying local inverter logs via FastMCP, the recommended setup includes:

1. **Primary Provider Choice**: Deploy **Google AI Studio (Gemini 2.0 Flash)** as the primary LLM engine. It offers an effective combination of zero cost, generous free quotas (1,500 RPD / 15 RPM / 1M TPM), native tool calling support, and fast generation speeds.
2. **Secondary Fallback Choice**: Integrate **GroqCloud (Llama 3.3 70B)** as an automated secondary fallback endpoint. If Gemini API endpoints experience temporary maintenance or rate limiting, the orchestrator script can automatically redirect requests to Groq.
3. **Local Server Setup**: Retain the Python FastMCP server (inv_mcp.py) running locally alongside inverter_logs.db. Host it in read-only mode (mode=ro) to safeguard data integrity while exposing structured endpoints (get_schema, execute_query) to the Python orchestrator script.
4. **Resilience Configuration**: Implement client-side exponential backoff and multi-project API key rotation in the Python script to manage rate limits seamlessly and maintain continuous zero-cost operation.

Works cited

1. Understanding Gemini 2.0 Pricing: A Guide to Google's AI Model Costs, https://neuroflash.com/blog/gemini-2-0-pricing/ 2. What Are Google AI Studio's Free Tier Limits and Rate Limits in 2026, https://docs.bswen.com/blog/2026-03-23-google-ai-studio-free-tier-limits/ 3. Free AI Platforms Daily Limits 2026: Gemini, Claude, OpenCode, https://www.mangomindbd.com/blog/free-ai-platforms-daily-limits-2026 4. 5 Free LLM API Providers You Can Use in 2026 - KDnuggets, https://www.kdnuggets.com/5-free-llm-api-providers-you-can-use-in-2026 5. Groq API Pricing

2026: Per-Token Cost Compared - Activepieces, https://www.activepieces.com/blog/groq-api-pricing-2026-per-token-cost-compared 6. Groq AI: LPU Chips, GroqCloud, and Pricing (2026) - Voiceflow, https://www.voiceflow.com/blog/groq 7. Best Free LLM APIs in 2026: Verified Limits Compared, https://decodethefuture.org/en/best-free-llm-apis-2026/ 8. SLMS FOR NATURAL LANGUAGE DATABASE INTERACTION, https://pdfs.semanticscholar.org/bf78/fb0b915e8208ad2f7c5fe8c60ac001e50569.pdf 9. Qwen2.5-Coder Technical Report - arXiv, https://arxiv.org/pdf/2409.12186 10. SQL-Zero: Self-Evolving Text-to-SQL - arXiv, https://arxiv.org/pdf/2609.04697 11. Rate limits | Gemini API - Google AI for Developers, https://ai.google.dev/gemini-api/docs/rate-limits 12. Running Pi with local LLMs on a Raspberry Pi sounds chaotic, but it, https://www.xda-developers.com/running-pi-with-a-local-llm-on-a-raspberry-pi-actually-works/ 13. Bypassing Gemini API Rate Limits with Smart Key Rotation in Next.js, https://medium.com/@entekumejeffrey/bypassing-gemini-api-rate-limits-with-smart-key-rotation-in-next-js-8acdee9f9550